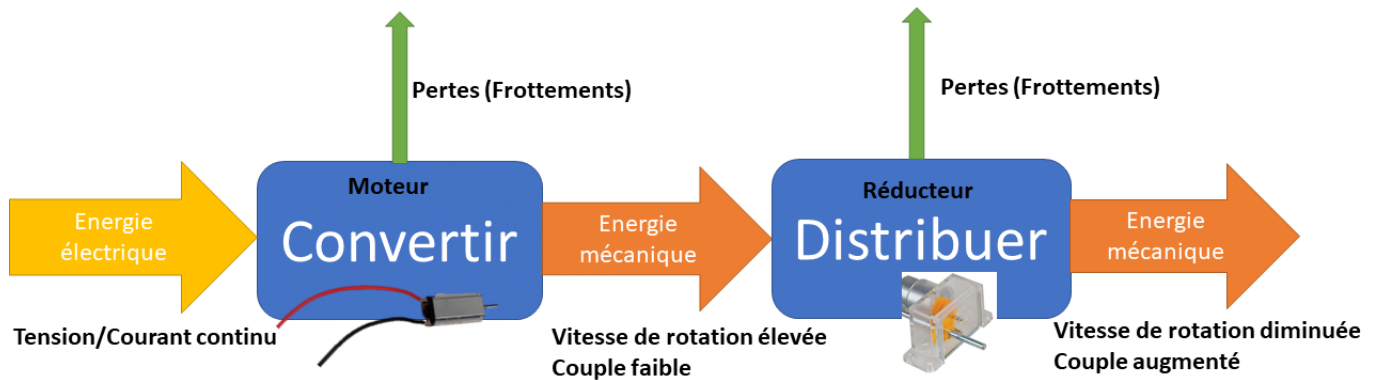


Commande de moteurs à courant continu

1. Principe de fonctionnement d'un moteur à courant continu

- Un moteur à courant continu **converti** une énergie électrique (Tension/Courant continu) en une énergie mécanique (mouvement de rotation de son axe de sortie)
- On ajoute parfois un réducteur (Ensemble d'engrenages mécaniques) à la sortie du moteur afin de réduire la vitesse de rotation et **augmenter le couple du moteur**.
- L'ensemble Moteur + Réducteur est appelé **motoréducteur**.



Rapport de réduction :

Pour 120 Tours en entrée -> 1 Tour en sortie

Exemple de moteur trouvé sur un site marchand :

DC MOTOR, MICRO MOTOR, 1.5V-3V 0.35A 16000RPM
PID# 17585
CA\$1.50 EACH

Energie d'entrée
(Tension/Courant)

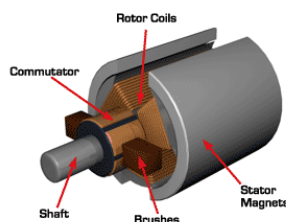
Energies de sortie
(Nombre de tours/min et couple)

Exemple de caractéristiques d'un bloc motoréducteur

Caractéristiques:

- Alimentation: 3 à 6 Vcc (nominal: 4,5 Vcc)
- Courant de blocage (sous 4,5 Vcc): 800 mA
- Couple: 1,4 kg.cm
- Réduction: 120:1
- Longueur de l'axe: 10 mm
- Diamètre l'axe: 3 mm (avec méplat)
- Dimensions: 36,5 x 29 x 22,5 mm

- Un moteur à courant continu comprend un **stator** constitué d'**aimants fixes**, un **rotor** constitué de **bobines** de cuivres, c'est la partie mobile, deux **broches** qui permettent d'injecter le courant dans le rotor et d'**un axe de sortie**.

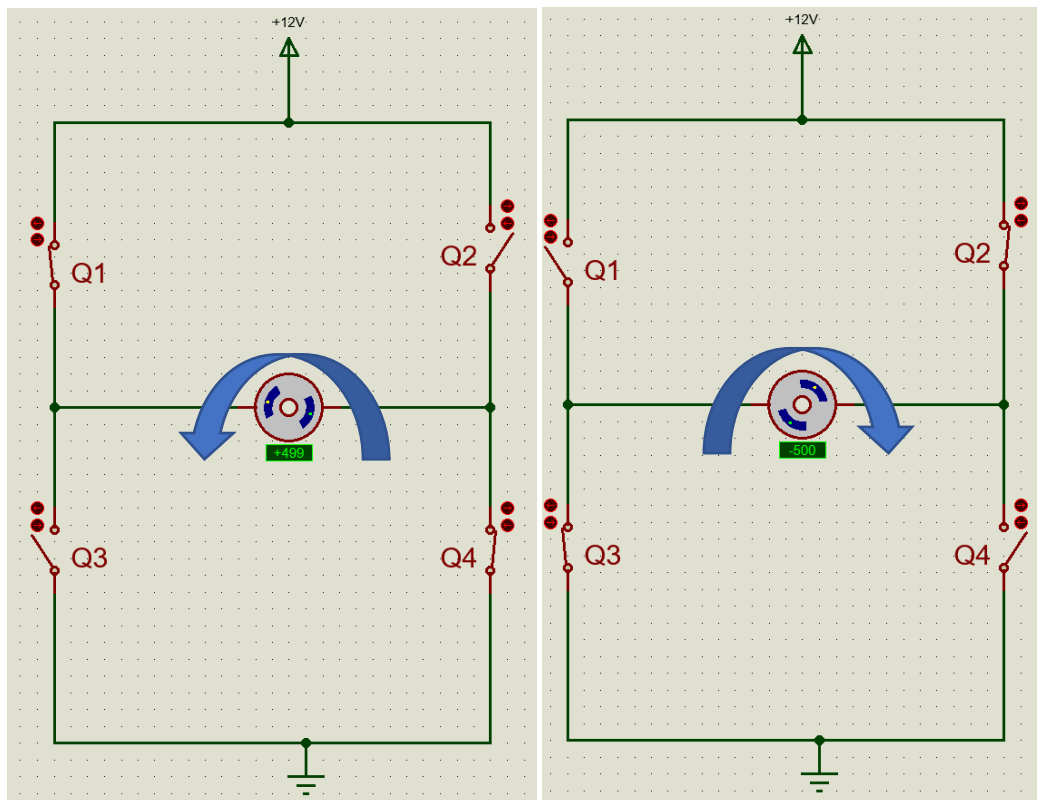


- Lorsque les bobines du rotor sont alimentées par un courant électrique (injecté par les 2 broches), elles se transforment en électroaimants et génèrent un champ magnétique.

- La combinaison des champs magnétiques du rotor et du stator engendre une force qui entraîne la rotation de l'axe de sortie.
- La tension d'alimentation d'un moteur va permettre de faire varier sa vitesse (il faut cependant respecter la tension nominale du moteur pour ne pas le détériorer).
- Le sens du courant dans le rotor déterminera le sens de rotation du moteur.
- Les moteurs à courant continu demandent généralement un courant trop important (>500mA) pour pouvoir être commandés directement par les sorties d'une carte Arduino. Il faut donc un montage électronique adapté pour pouvoir réaliser cette opération.

2. Principe du Pont en H

Le pont en H est un montage électronique qui permet d'inverser le sens du courant dans un moteur et donc son sens de rotation.



3 Configurations possibles

Sens 1 : Q1 et Q4 Fermé / Q2 et Q3 Ouverts

Sens 2 : Q1 et Q4 Ouverts / Q2 et Q3 Fermés

Freinage : Q1 et Q2 Fermés ou Q3 et Q4 Fermés

3. Les Transistors

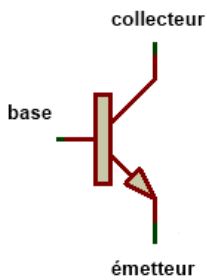


Dans le pont en H, ce ne sont pas des interrupteurs qui sont utilisés mais des **transistors** utilisés en régime de saturation.

Le transistor est un composant électronique inventé à la fin des années 1940 et qui est à la base du développement de l'électronique et de l'informatique. Les microprocesseurs de nos ordinateurs et de nos smartphones sont notamment composés de millions de transistors

miniaturisés.

Un des usages du transistor (il y en a d'autres) et le « régime de saturation », dans ce cas on l'utilise pour laisser ou non passer le courant dans un circuit électrique. C'est une sorte d'interrupteur que l'on peut commander avec un courant électrique.



Les 3 bornes du transistor sont nommées Emetteur, Base et Collecteur. En régime de saturation, on injecte un courant dans la base lorsque l'on veut fermer la liaison Collecteur/Emetteur.

Dans notre cas, ce sont les **sorties de l'Arduino qui seront utilisées pour injecter du courant dans la base** et ainsi ouvrir/fermer la liaison Collecteur/Emetteur.

État bloqué

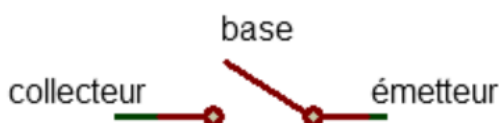


schéma équivalent du transistor à l'état bloqué

État saturé

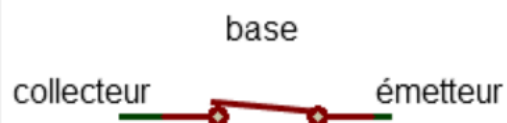
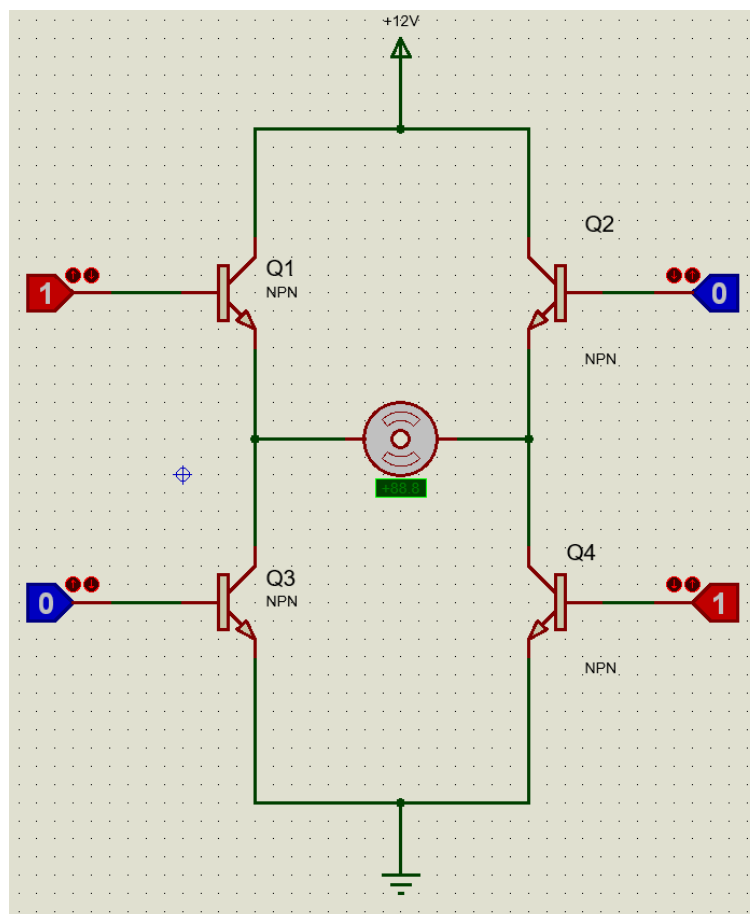


schéma équivalent du transistor à l'état saturé

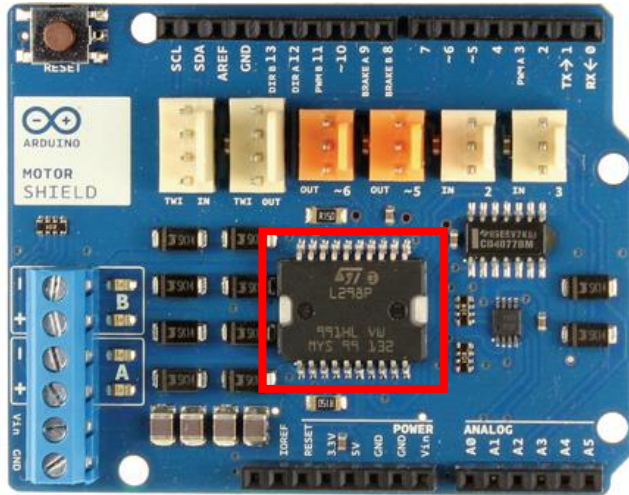
(Source : openclassroom)

Un pont en H avec des transistors peut donc être réalisé de cette manière :

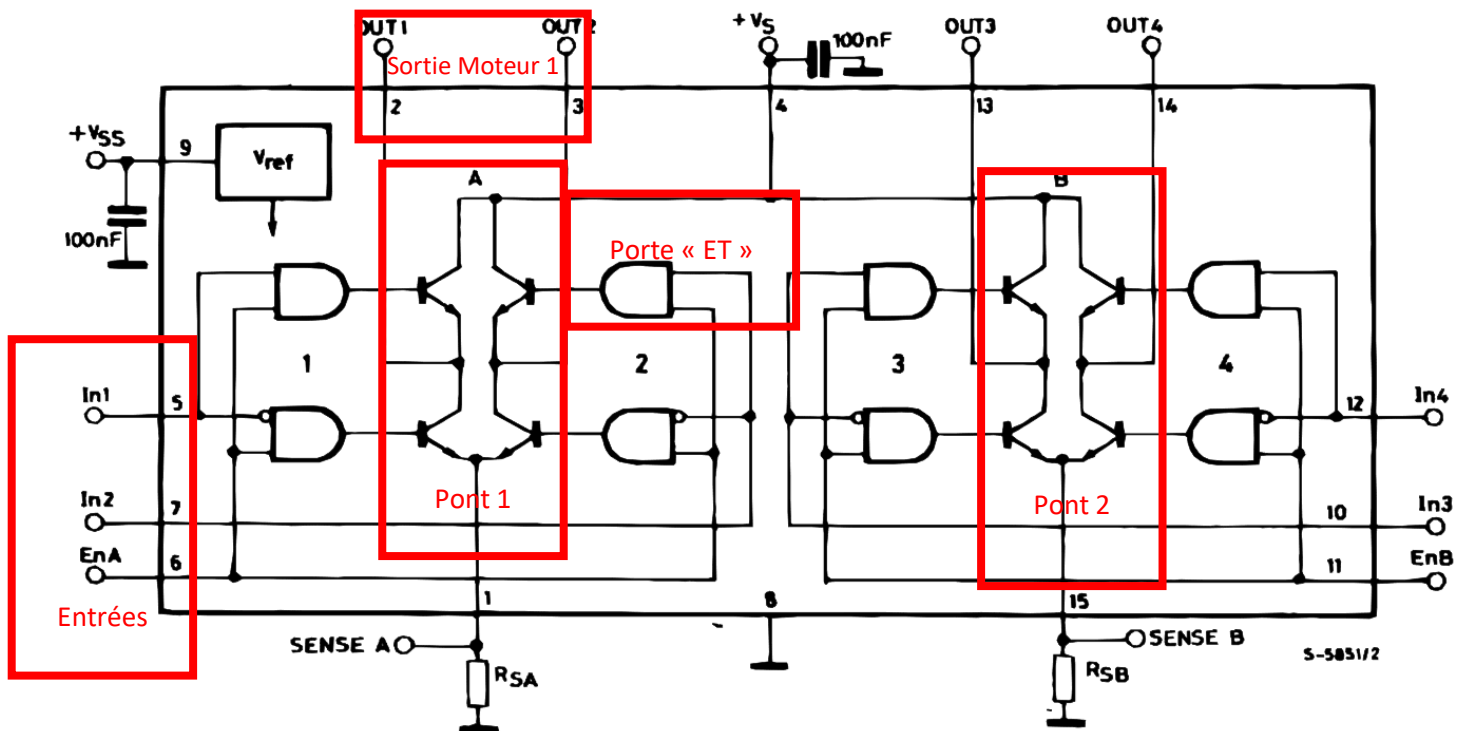


Les états logiques (0 ou 1) sur les bases des transistors pouvant être générés par les sorties d'un microcontrôleur (Arduino par exemple)

4. Fonctionnement du L298

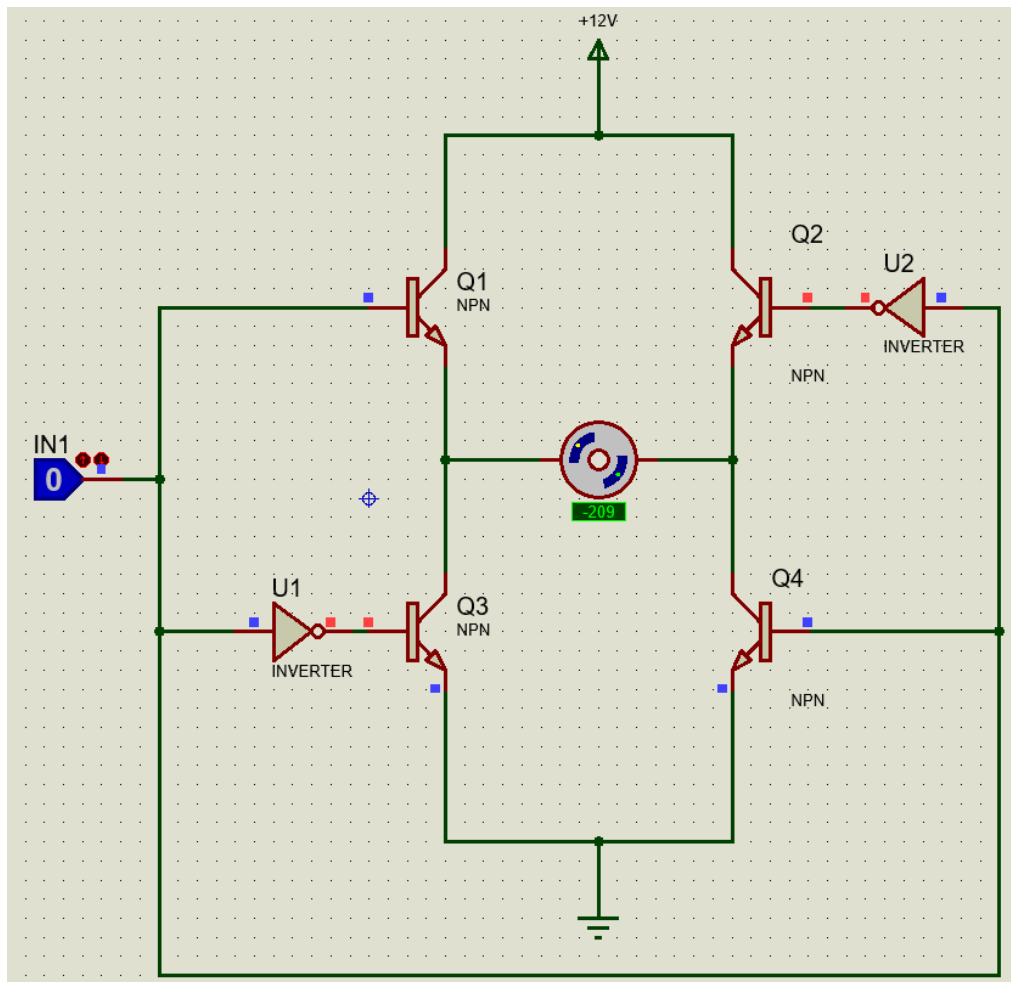


Lorsque l'on achète un Shield moteur pour Arduino, il est généralement équipé d'un circuit intégré appelé L298. Ce circuit n'est rien d'autre qu'un **double pont en H miniaturisé**, on peut donc grâce à lui commander 2 moteurs dans les 2 sens. Pour informations, voici le schéma équivalent de ce circuit :



On peut notamment y repérer les 2 **pont en H** réalisés avec des **transistors** et des **portes logiques « ET »** qui permettent de commander simultanément plusieurs transistors et de disposer d'une entrée **Enable** dont nous parlerons plus tard.

Sur la plupart des Shields que l'on peut trouver, pour limiter le nombre de sorties nécessaires pour commander un moteur, une astuce a été utilisée, on pourrait la résumer comme ceci :



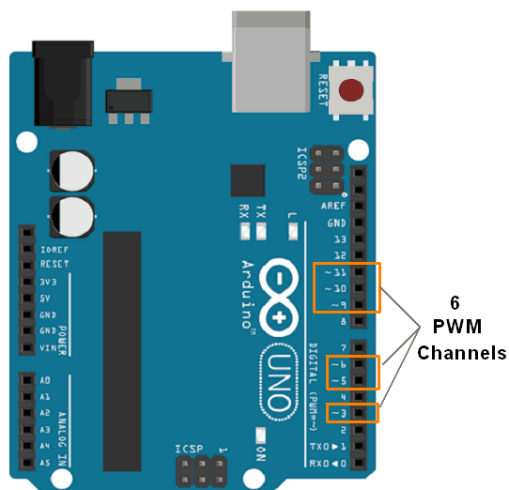
Une seule Entrée (IN1) permet de commander les 4 transistors, des portes inverseuses permettent d'inverser le niveau logique sur Q2 et Q3.

Ce qui donne

IN1	Etat des transistors
0	Q1 et Q4 Ouvert / Q2 et Q3 Fermés -> Sens 1
1	Q1 et Q4 Fermés / Q2 et Q3 Ouvert -> Sens 2

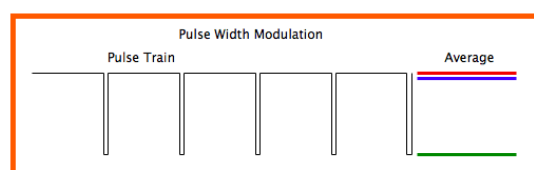
5. Commande par PWM

La **Pulse Width Modulation** ou Modulation de Largeur d'Impulsion (MLI) en français est une technique qui consiste à faire commuter (passage de l'état haut à l'état bas) à vitesse rapide (490 ou 980 Hz pour l'Arduino) une sortie pour ainsi faire varier la tension moyenne du signal.



Toutes les sorties de la carte Arduino ne permettent pas cette fonctionnalité. Les sorties PWM sont indiqués par le symbole ~.

Dans le cadre de la commande de moteurs, la PWM permet de commander par intermittence les transistors qui ouvrent et ferment l'alimentation du moteur et ainsi faire diminuer sa vitesse.



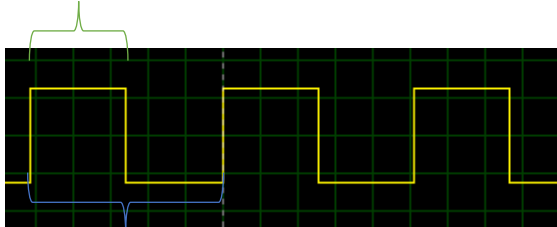
La commande des sorties PWM sur Arduino se fait avec l'instruction

`AnalogWrite(Broche, Valeur)`

Valeur doit être compris entre 0 et 255. 255 Correspondant à un état Haut de la sortie 100% du temps.

Pour la PWM, on appelle le rapport $\alpha = \frac{\text{Temps à l'état Haut}}{\text{Période du signal}}$ le **rapport cyclique** du signal. (Souvent nommé Alpha : α)

Temps état Haut

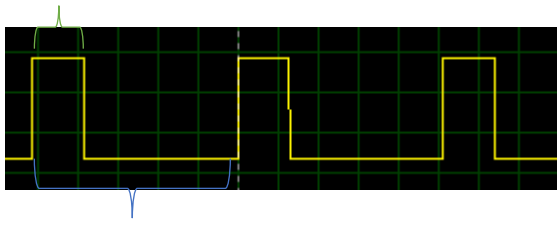


Ainsi avec la commande `AnalogWrite(5, 127)`

J'aurais donc sur ma sortie 5 un signal à 50% à l'état Haut et 50% à l'état Bas.

Rapport Cyclique : $\alpha = 50\%$

Temps état Haut



De même si je fais : `AnalogWrite(5, 64)` ;

Le Signal de sortie sur la broche 5 est à 25% du temps à l'état HAUT.

Rapport Cyclique : $\alpha = 25\%$

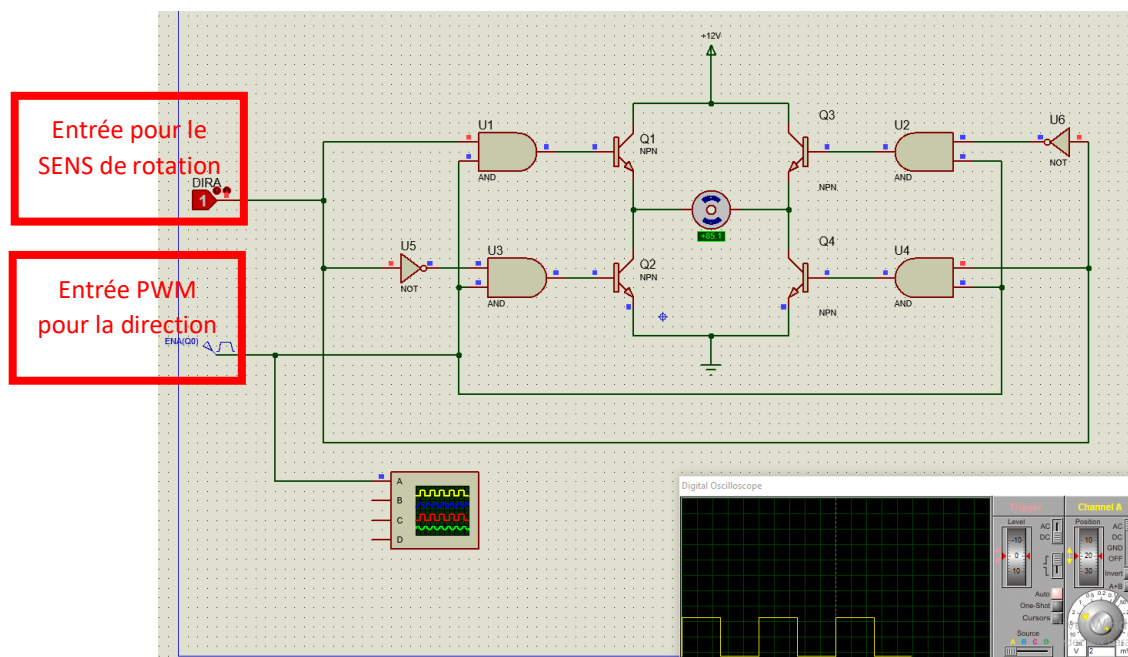
Si j'envoie ce signal sur la **base** de mes transistors, ceux-ci vont donc s'ouvrir et se fermer selon le rapport cyclique programmé. La vitesse de rotation du moteur sera donc égale à :

$$VitesseMoteur = Vitesse\ nominale \times \alpha$$

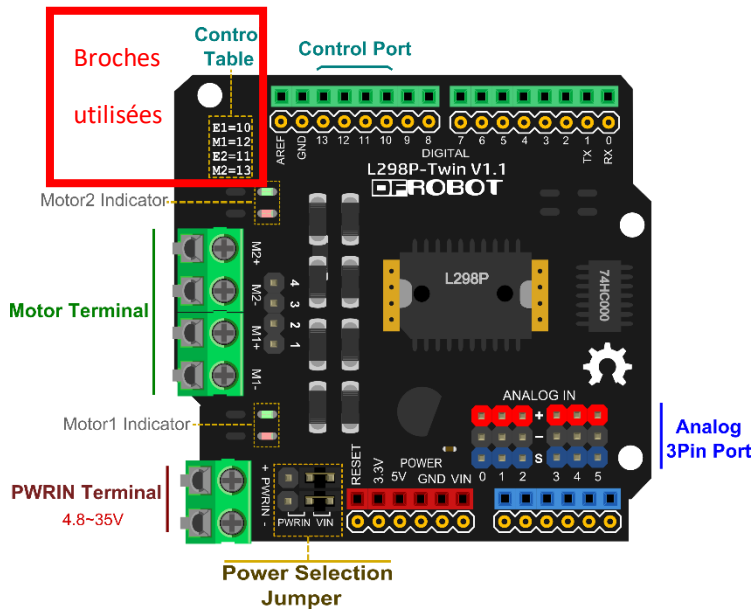
La vitesse nominale étant liée à la tension d'alimentation du moteur (Qui doit être la tension nominale indiquée dans la documentation technique du moteur.

Cas des Shields moteurs et du L298:

Le montage du L298 permet grâce au câblage de portes logiques d'avoir une entrée commandant le sens de rotation (souvent nommée IN1 ou DIRA) et une entrée PWM pour la vitesse (nommée PWMA ou PWM1). On peut schématiser le câblage du Shield moteur (pour 1 seul moteur) comme ceci :



6. Programmation Arduino



Une fois que l'on sait tout cela, la commande des moteurs est assez simple, il suffit de savoir sur quelles sorties de l'Arduino sont reliées les Entrées DIRA et DIRB et PWMA et PWMB. Cette information se trouve dans la documentation du Shield. Certains Shields proposent des cavaliers permettant de choisir les sorties, on peut ainsi superposer les Shields sur la carte et commander jusqu'à 6 moteurs.

Dans le cas du shield ci-contre, E1=10 La sortie vitesse du moteur 1 (En PWM donc) sera donc à programmer sur D10. Le sens de rotation du moteur 1 sera à programmer sur la sortie D12 (M1=12). Pour le moteur 2 ce seront les broches 11 (PWM) et 13 (Sens).

On peut donc programmer le moteur comme ceci :

```

1 //Définition des constantes
2
3 #define DIRA 12 // Direction du moteur A
4 #define PWMA 10 // Vitesse du moteur A
5 #define DIRB 13 //Direction du moteur B
6 #define PWMB 11 //Vitesse du moteur B
7 void setup() {
8
9 //Déclaration des broches en sorties
10 pinMode(DIRA, OUTPUT);
11 pinMode(PWMA, OUTPUT);
12 pinMode(DIRB, OUTPUT);
13 pinMode(PWMB, OUTPUT);
14
15 // On éteins les 2 moteurs
16 analogWrite(PWMA, 0);
17 analogWrite(PWMB, 0);
18 }
19
20 void loop() {
21 // On envoie les deux moteurs à pleine vitesse dans le sens 1
22 digitalWrite(DIRA,HIGH);
23 digitalWrite(DIRB,HIGH);
24 analogWrite(PWMA,255);
25 analogWrite(PWMB,255);
26 delay(2000);
27 //On envoie les deux moteurs à 50% de la vitesse dans le sens 2
28 digitalWrite(DIRA,LOW);
29 digitalWrite(DIRB,LOW);
30 analogWrite(PWMA,127);
31 analogWrite(PWMB,127);
32 delay(2000);
33 }

```